Project ID: 22-26/CSIT/G27

A PROJECT REPORT ON

NeuroNet Diagnostics

Submitted
for the Partial fulfillment of award of
Bachelor of Technology
in
Computer Science And Information Technology
by

Anurag Singh (2100270110016.)

Akshika Agrawal (2100270130022.)

Kavyansh Srivastava (2100270110042.)

Under the guidance of

Dr. Nandita Goyal

AJAY KUMAR GARG ENGINEERING COLLEGE,

GHAZIABAD

November 9, 2025

Declaration

We here by declare that the work presented in this report, entitled as **NeuroNet Diagnostics**, was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors and sources for all the words, ideas, diagrams, graphics, computer programs, experiments, and results that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors and sources.

We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, We shall be fully responsible and answerable.

Name : Anurag Singh
Roll No. : 2100270110016

Name : Akshika Agrawal
Roll No. : 2100270130022

Name : Kavyansh Srivastava
Roll No. : 2100270110042

Certificate

This is to certify that the report entitled **NeuroNet Diagnostics** submitted by **Anurag Singh (2100270110016), Akshika Agrawal (2100270130022), Kavyansh Srivastava (2100270110042)** to the Dr. A. P. J. Abdul Kalam Technical University, Lucknow (U.P.) in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Information Technology is a bonafide record of the project work carried out by him/her under my/our guidance and supervision. This report in any form has not been submitted to any other university or institute for any purpose, to the best of my knowledge.

Dr. Nandita Goyal
Associate Professor
Department of Information Technology
Ajay Kumar Garg Engineering College

Dr Rahul Sharma
Professor & HOD
Department of Information Technology
Ajay Kumar Garg Engineering College

Place: Ghaziabad
November 9, 2025

Acknowledgements

We would like to express our thanks to all the people who have helped bring this project to the stage of fulfillment. We would wish to put on record very special thanks to my major project mentor, **Dr. Nandita Goyal**, for the support, guidance, encouragement, and some very valuable insight that she guided us in the entire process. Her mentorship has been very pivotal in terms of shaping of our project and leading us toward excellence.
We would like to appreciate our Head of the Department, **Dr. Rahul Sharma**, who had provided us with the wherewithals and put us into an environment that would bring out such innovation towards learning. We would also want to appreciate our teachers and faculty members for all that they share at this crucial juncture in our academic career. We wish to appreciate many more for: helped out or, with their presence, indirectly made contributions to this project.

Contents

Declaration **i**

Certificate **ii**

Acknowledgements **iii**

List of Figures **vi**

1 Introduction **1**
1.1 Problem Statement of Project 1
1.2 Scope of Project 2
1.3 Detail of Problem Domain 3
1.4 Gantt Chart 4
1.5 System Requirements 5
1.6 Project Report Outline 6

2 Literature Review **7**
2.1 Related Study 7
2.2 Research Gaps 11
2.3 Objective of Project 12

3 Methodology Used **13**

4 Designing of Project **18**
4.1 0-Level DFD 18
4.2 1-Level DFD 19
4.3 2-Level DFD 21
4.4 Use Case Diagram 23

5 Detailed Designing of Project **25**
5.1 Class Diagram 25
5.2 Entity-Relationship Diagram (ERD) 27
5.3 Activity Diagram 29
5.4 Sequence Diagram 31

Appendix A **33**

Appendix B **34**

Appendix C **35**

List of Figures

1.1 Gantt Chart 4

3.1 Flow Diagram of Proposed Realtime Collaborative Code Editor 14

4.1 0-level DFD 18
4.2 1-level DFD 20
4.3 2-level DFD 22
4.4 Use Case Diagram 23

5.1 Class Diagram 26
5.2 ER Diagram 28
5.3 Activity Diagram 30
5.4 Sequence Diagram 31
5.5 Screenshot Of Application 33
5.6 Screenshot Of Application 33
5.7 Research paper 34
5.8 Plagiarism Report 35
5.9 AI Report 36

Chapter 1

Introduction

1.1 Problem Statement of Project

In today's digital era, programming is considered a fundamental skill across disciplines, but its teaching methods in schools and colleges remain outdated. While programming languages like Python, Java, and C++ are included in the curriculum, the approach is largely theoretical. Students are often made to write code on paper or use outdated labs instead of practicing in live coding environments. This gap results in a lack of real-world exposure, weak debugging skills, and poor readiness for industry expectations.

Another major issue is the infrastructural challenge faced by many institutions. Computer labs rely on resource-heavy installations, frequent updates, and high maintenance costs, making them inaccessible to schools with limited facilities. Students are discouraged when they face slow systems or technical barriers before even beginning to code. These challenges restrict experimentation and creativity, further widening the gap between learning and practical application.

The absence of collaboration also limits learning outcomes. Current tools and IDEs are designed for individual use, ignoring the teamwork-based nature of modern software development. Students rarely get exposure to working in groups, sharing ideas, and debugging collaboratively, which are essential skills in the professional world. At the same time, teachers lack tools to monitor students' progress in real time or provide guided, interactive coding support during classroom sessions.

Finally, affordability and inclusivity remain critical concerns. Many advanced coding platforms require costly subscriptions or high-end systems, which excludes students from underprivileged backgrounds. Furthermore, existing solutions ignore healthy technology practices, leading

to unmonitored screen time and distractions. There is thus a strong need for a lightweight, browser-based, collaborative code editor that works on low-end devices, promotes teamwork, supports teacher–student interaction, and integrates AI assistance for real-time guidance. Such a platform would address accessibility, inclusivity, and pedagogical gaps in current programming education.

1.2 Scope of Project

1. **Accessible Coding Platform** The project will deliver a browser-based, lightweight editor that requires no installation. It will run efficiently even on low-end systems, making coding accessible in resource-limited institutions. This ensures every student can practice coding without technical or financial barriers.

2. **Real-Time Collaboration** The platform will allow multiple users to code, edit, and debug simultaneously. Students can experience teamwork, peer learning, and collaborative problem-solving. This reflects modern software development practices and prepares learners for industry roles.

3. **Teacher Integration** Teachers will be able to create coding sessions and monitor student activities in real time. They can give instant feedback and guide students during classroom practice. This improves classroom engagement and ensures effective instructional support.

4. **AI-Powered Assistance** The editor will include AI features like contextual hints, code suggestions, and debugging help. It will act as a virtual tutor, enabling students to learn independently. Combined with teacher support, it creates a dual guidance system for learners.

5. **Inclusivity and Future Growth** The project emphasizes affordability, inclusivity, and healthy digital practices. Screen-time monitoring and safe communication will be integrated for responsible use. In the future, the system can expand to hackathons, coding competitions, and professional collaboration.

1.3 Detail of Problem Domain

1. **Educational Technology (EdTech) -** The project lies in the field of EdTech, where technology is used to enhance learning. Programming education is a major focus area, but current tools are outdated or limited. A modern, collaborative platform is needed to improve student engagement and outcomes.

2. **Computer Science Education -** Traditional programming courses emphasize syntax and theory but lack practical exposure. Students are rarely introduced to teamwork-based coding or real-time problem-solving. This gap between curriculum and industry demands defines a critical challenge in the domain.

3. **Collaborative Software Development -** In the professional world, coding is rarely an individual task—it is team-driven. Most schools and colleges, however, do not provide students with such experiences. A domain challenge is designing platforms that bring collaborative practices into education.

4. **AI-Assisted Learning -** Artificial Intelligence is transforming education by providing personalized learning support. However, its adoption in coding education remains minimal at the school and undergraduate level. Integrating AI into a collaborative editor bridges this gap in the problem domain.

5. **Accessibility and Inclusivity in Learning -** Many existing coding platforms require costly subscriptions or high-end devices. This creates barriers for institutions and students from weaker socio-economic backgrounds. The domain challenge is to create an affordable, lightweight solution that reaches all learners.

1.4 Gantt Chart

A Gantt chart is a visual project management tool that displays tasks, their duration, and dependencies on a timeline. It helps in tracking progress, scheduling tasks, and improving team coordination.

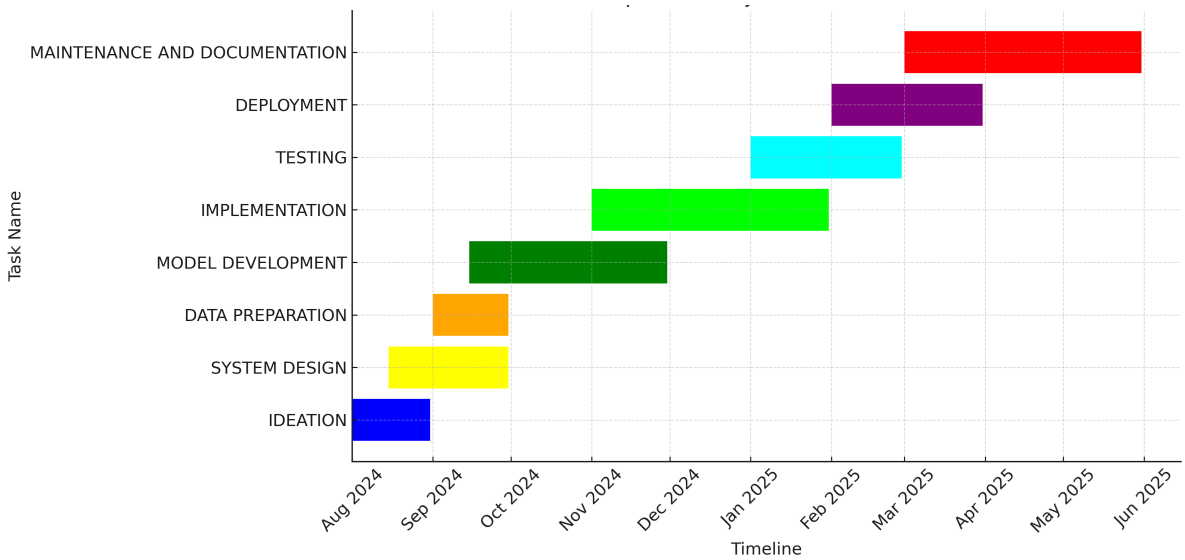

Figure 1.1: Gantt Chart

1.5 System Requirements

Hardware Requirements :

1. **Processor:** Minimum Intel i3 or equivalent, Recommended Intel i5/i7 for smooth multitasking.
2. **RAM:** Minimum 4 GB, Recommended 8 GB or above for handling real-time collaboration.
3. **Storage:** At least 250 GB HDD/SSD, with 1 GB free space for project-related files.
4. **Display:** Minimum resolution 1366 × 768, Recommended Full HD for better UI experience.
5. **Internet:** Stable broadband connection (minimum 2 Mbps, recommended 10 Mbps).

Software Requirements :

1. **Operating System:** Windows 10 or higher / Linux / macOS (cross-platform).
2. **Programming Languages:** JavaScript (for frontend & backend), Node.js runtime.
3. **Frameworks/Libraries:** React.js for frontend, Express.js for backend, Socket.IO or WebRTC for real-time communication.
4. **Database:** MongoDB or Firebase (for storing user data and projects).
5. **Editor/IDE Support:** Browser-based Monaco Editor (used in VS Code).
6. **AI Support:** Integration with OpenAI/LLM APIs for chatbot assistance.
7. **Others:** Web browser (Google Chrome/Firefox/Edge).

1.6 Project Report Outline

Chapter 2 – Literature Review

Discusses related studies, existing solutions, and research gaps. Highlights the need for the proposed system and justifies its development.

Chapter 3 – Methodology Used

Explains the approach, system architecture, diagrams (class, use case, ERD), and tools used. Describes how the project requirements are translated into a working solution.

Chapter 4 – Designing of Project

Focuses on detailed design aspects including UI/UX, flowcharts, and screen designs. Demonstrates how each module and component is structured and implemented.

Chapter 5 – Detailed Designing of Project

Provides an in-depth explanation of functionalities, coding logic, and workflows. Includes detailed explanations of real-time collaboration, AI integration, and other key features.

Chapter 6 – Results and Discussion

Presents screenshots, outputs, and discussions of the implemented system. Analyzes how the objectives have been achieved and evaluates performance.

Chapter 7 – Further Work

Discusses possible enhancements, future extensions, and improvements for the system.

Appendices

Includes supplementary materials such as code snippets, diagrams, and additional references.

Chapter 2

Literature Review

2.1 Related Study

Aditya Kurniawan et al. (2015) [1]: This paper introduces CodeR, a web-based real-time code editor application designed to facilitate collaborative programming. Utilizing WebSocket technology, CodeR enables users to write, execute, and display code results through a terminal, all within a shared workspace. The application supports real-time collaboration, chat features, and terminal building, catering to programming languages such as C, C++, and Java. The study emphasizes the importance of web-based tools in modern software development, allowing developers to collaborate seamlessly regardless of location.

Shivam Chowdhury et al. (2025) [2]:The research focuses on the design and development of a real-time code editor application aimed at enhancing collaboration during technical interviews. By employing WebSocket technology, the application allows users to collaborate in real-time on coding tasks. It features a workspace for creating, running, and building source code, along with real-time chat and terminal functionalities. The study highlights the potential of such platforms in remote technical assessments, providing an efficient and accessible environment for both interviewers and candidates.

Khushwant Virdi et al. (2023) [3]: This paper delves into the evolution and challenges of collaborative code editors, particularly in the context of real-time multi-user coding and knowledge sharing. The authors review the underlying technologies and protocols employed in these platforms, such as WebSocket communication and Operational Transformation (OT) algorithms. They identify persistent issues like latency, synchronization challenges, and conflict resolution complexities. The study also presents a

prototype collaborative code editor, evaluating its performance and usability compared to existing platforms, and offers recommendations for future research and development in this field.

Mohd Saad Ansari (2025) [4]: The study presents a minimalistic, browser-based real-time collaborative code editor built using the MERN stack and Socket.IO for real-time communication. The platform enables multiple users to write, view, and execute code together seamlessly, making it suitable for peer coding, interviews, and learning scenarios. The research emphasizes the importance of simplicity and performance in designing collaborative tools, aiming to provide an intuitive and efficient environment for real-time programming collaboration.

Anup Lal Yadav et al. (2024) [5]: This research explores the design and development of an advanced collaborative code editor prototype that integrates WebSocket communication and OT algorithms to facilitate real-time multi-user coding. The authors conduct a comprehensive literature review to understand the evolution of collaborative code editors and identify key technologies and protocols. They also address challenges such as latency and synchronization issues, proposing solutions to enhance collaboration and knowledge sharing among distributed development teams.

Krishna Aher Prof. Jagruti R. Mahajan (2023) [6]: This paper presents the development of a real-time collaborative code editor web application using React.js, Node.js, and Socket.io. The application enables developers to collaborate on code in real-time, regardless of their location. Key features include syntax highlighting, automatic code formatting, code versioning, and error highlighting. The study emphasizes the importance of such tools in enhancing productivity and collaboration among development teams. Future prospects include integrating artificial intelligence (AI) for intelligent code suggestions and auto-completion, machine learning (ML) algorithms for automatic error detection and fixing, and virtual reality (VR) or augmented reality (AR) for immersive coding experiences.

Anup Lal Yadav et al. (2023) [7]:This research paper delves into the intricate fabric of collaborative code editing, aiming to unravel the underlying technologies, challenges, and advancements in this field. The study begins with a development in this field, emphasizing the need for continued innovation and the exploration of novel strategies to address

the persistent challenges faced in collaborative code editing environments. The authors discuss key technologies such as WebSocket communication and Operational Transformation (OT) algorithms, and propose solutions to enhance collaboration and knowledge sharing among distributed development teams.

R. Pathak et al. (2024) [8]: The authors explore the trends, advantages, and future prospects of coding and collaborative applications, demonstrating the transformative potential of real-time tools in computer development. The study introduces V-Code, an advanced application for remote coding using technologies such as Nuxt.js and Python Flask. V-Code incorporates OpenAI machine learning models to provide intelligent code suggestions and auto-completion, enhancing the coding experience. The research highlights the growing importance of real-time collaborative code editors in modern software development.

Table 2.1: Literature Review

S.NO.	Author	Year	Publisher	Methodology	Result	Limitations
[1]	Oussama El Gannour , et.al	2022	IJACSA	CNN	96%	limited dataset size,MRI image quality issue, and lack of sophisticated image modification techniques.
[2]	A. S. Prakaash , , et.al	2022	IJEER	DRLSTM	95.67%	High computing time, intricate mathematical calculations, and lower learning rate for model training
[3]	Muhammad Khan , et.al	2022	APPLIED ARTIFICIAL INTELLIGENCE	J48, MB and RT	96.8%,95.5% and 94.8%	small datasets, poor generalization across tumor types, sensitivity to noisy data, and insufficient annotated medical data
[4]	Pallab K. Choudhary, et.al	2022	IEEE	VGG-19-SVM	98.39%	small sample size and lack of diversity.
[5]	Nosheen Sohail , et.al	2022	IEEE	DEEP LEARNING	91.34%	limited sample size and potential bias.
[6]	Shubhangi Solanki , et.al	2023	IJISAE	CNN	97.39%	small sample size and short follow-up duration.
[7]	Sohaib Asif , et.al	2022	IEEE	XCEPTION(GANs)	91.68%	small sample size and lack of multicenter data.
[8]	Md. Saikat Islam Khan, et.al	2022	NIH	23-LAYER CNN	97.8%	small sample size, lack of diversity, and reliance on retrospective data analysis.
[9]	Rafiqul Islam, , et.al .	2023	SCIRP	CNN and KSVM	97.4%	small sample size, lack of multicenter data, and potential measurement errors.

2.2 Research Gaps

Despite several studies and developments in collaborative coding platforms and intelligent code editors, there remain critical gaps that justify the need for further research:

1. Limited Real-Time Collaboration for Students : Existing platforms primarily focus on professional developers or general-purpose collaborative editors. There is insufficient research on designing collaborative coding environments specifically tailored for school or college students, addressing their learning pace and programming comprehension.

2. Integration of AI Assistance : While AI-based code suggestion tools like GitHub Copilot exist, few studies integrate AI-assisted learning features directly within a real-time collaborative environment to guide students or novice coders during live coding sessions.

3. Cross-Platform Accessibility : Most solutions either focus on web-based or desktop-based editors. There is a lack of seamless cross-platform real-time editors that synchronize edits across devices without lag or data inconsistency.

4. Scalability and Performance Evaluation : Existing research often neglects detailed evaluation of scalability for multiple concurrent users. Handling real-time synchronization for a large number of students in a classroom setting remains an underexplored area.

5. User Engagement and Interaction Analysis : Few studies examine how collaborative coding platforms impact student engagement, peer interaction, and learning outcomes in real-time coding environments. Quantitative and qualitative assessments in academic settings are limited.

6. Security and Privacy in Educational Contexts : While collaboration tools exist, research on secure real-time coding for students, including data privacy, role-based access, and session management, is sparse.

7. Customization for Learning Objectives : Most collaborative editors do not provide adaptive interfaces or features that align with specific learning objectives, such as beginner-friendly hints, syntax highlighting for specific languages, or interactive debugging exercises.

2.3 Objective of Project

The primary objective of this study is to design and develop a Realtime Collaborative Code Editor that enhances coding skills and learning experiences for students. The study aims to address the limitations identified in the literature and research gaps. The objectives are as follows:

1. **Develop a Real-Time Collaborative Environment :** To create a web-based platform that allows multiple users to write, edit, and execute code simultaneously with minimal latency and real-time synchronization.
2. **Integrate AI-Assisted Learning Features :** To incorporate intelligent code suggestions, error detection, and hints to support novice learners and enhance coding efficiency.
3. **Ensure Cross-Platform Accessibility :** To design the editor such that it functions seamlessly across devices, including desktops, laptops, and tablets, without compromising performance.
4. **Evaluate User Engagement and Learning Outcomes :** To assess how collaborative coding affects student interaction, engagement, and overall learning, using qualitative and quantitative metrics.
5. **Maintain Security and Data Privacy :** To implement secure user authentication, role-based access, and data protection mechanisms suitable for educational environments.
6. **Provide Customizable Learning Tools :** To allow users and educators to customize the interface, programming languages, and learning modules according to specific course objectives.

Chapter 3

Methodology Used

Public Datasets : One should begin to look for public data-sets that have something to do with brain MRI scans.

This section describes the methodology proposed to develop a Realtime Collaborative Code Editor. The study aims to provide students with an interactive environment where multiple users can code simultaneously, receive AI-assisted guidance, and collaborate in real time. The system is designed to improve coding learning, foster teamwork, and provide immediate feedback on code execution.

The designed system takes input from users through a web interface and performs multiple stages, including real-time synchronization, AI assistance, syntax handling, and code execution. Each process is described

in detail below.

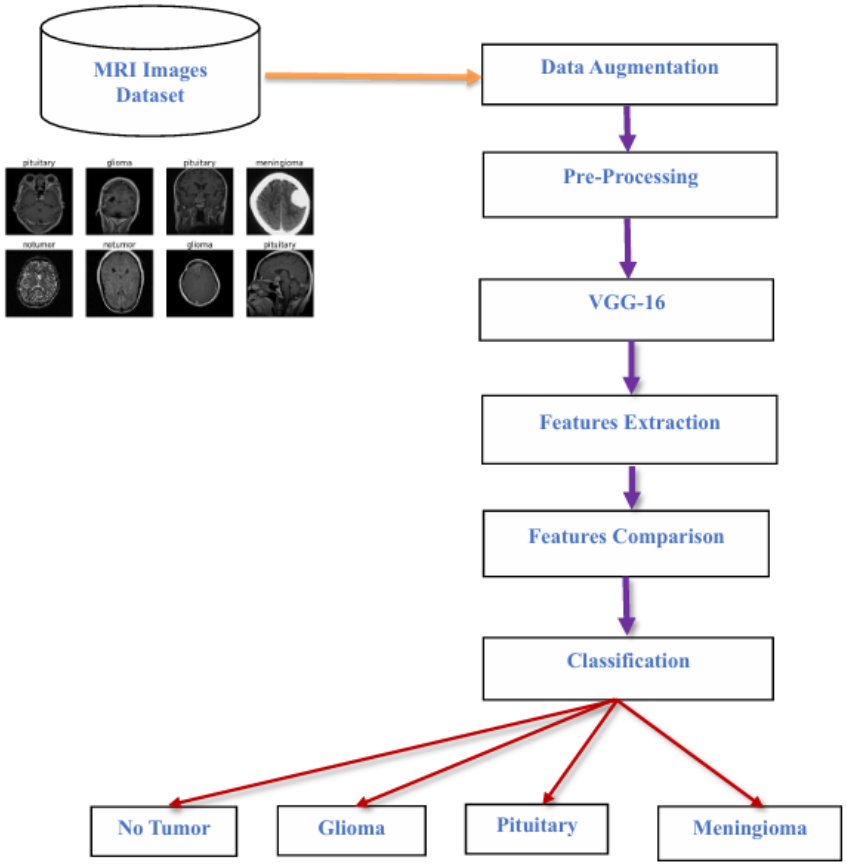

Figure 3.1: Flow Diagram of Proposed Realtime Collaborative Code Editor

3.1 Code Input and User Interaction

Code input is the first step in the proposed system. Users access the editor through a web-based interface and begin typing code in their preferred programming language (Python, Java, C++, etc.). The editor captures every keystroke and sends it to the server for real-time synchronization.

This process allows multiple users to work on the same code simultaneously without overwriting each other's changes. Features such as line numbers, autocomplete, and editor themes are included to enhance usability and accessibility.

Example: If three students are collaborating on a Python script, any change made by one student is instantly reflected in the editor windows of the other two students.

3.2 Real-Time Synchronization

Real-time synchronization is critical to maintaining consistency across multiple users. The system uses Conflict-Free Replicated Data Types (CRDTs) implemented via Yjs, combined with WebSocket or Socket.IO for low-latency communication.

- CRDT ensures that simultaneous edits do not conflict.
- WebSocket provides a two-way communication channel between the client and server.

  This method avoids traditional locking mechanisms, making collaboration seamless and reducing delays even when multiple users are editing the same line of code. Example: If one student adds a new function while another modifies a variable in the same script, CRDT merges the changes automatically without losing any edits.

3.3 Pre-processing and Syntax Handling

Before code can be executed or AI suggestions applied, it undergoes preprocessing:

- Syntax Highlighting: Differentiates keywords, variables, and functions using colors for easier reading.
- Error Detection: Identifies syntax errors in real time and highlights them to the user.
- Code Formatting: Ensures consistent indentation and spacing.

  Preprocessing not only improves readability but also enhances the accuracy of AI suggestions and prevents runtime errors.

  Example: Indentation errors in Python are flagged immediately, allowing students to correct them before execution.

3.4 AI-Assisted Code Suggestions

The AI module provides:

- Autocomplete for functions and variables
- Contextual code hints based on programming patterns
- Error detection and suggestions for correction

  The AI is designed to support novice learners by providing hints and examples without giving direct answers. This encourages learning

through exploration while reducing frustration caused by frequent coding errors.

Example: When writing a loop, the AI can suggest the correct syntax and possible conditions based on previously written code.

3.5 Feature Extraction and Analysis

Feature extraction involves identifying key components of the code:

- Functions and methods
- Loops and control structures
- Variable usage and scope

This information is used by the AI for predictive suggestions, collaborative analytics, and conflict resolution. It also helps in generating reports on individual or team performance during coding sessions.

Example: Tracking how often a student uses loops incorrectly can help the AI provide targeted hints for improvement.

3.6 Comparison and Conflict Resolution

Changes made by different users are compared and merged using CRDT. Conflict resolution ensures:

- No edits are lost
- All participants see the same version of the code
- Collaboration is smooth and uninterrupted

Example: If two users edit the same line simultaneously, CRDT merges the changes in a logical sequence without overwriting any edits.

3.7 Code Execution and Output Display

The system executes the code in a secure sandbox environment to prevent security risks. Outputs are displayed to all users in real time. Features include:

- Console output panel for program results
- Highlighting runtime errors

- Supporting multiple programming languages

  Example: When a Python script prints numbers from 1 to 10, all collaborators see the output instantly, and any error (like division by zero) is highlighted immediately.

3.8 Dataset

To evaluate the system, sample programs and exercises are used:

- Beginner-level programs (loops, conditional statements)
- Intermediate collaborative exercises
- Complex scripts for AI-assisted testing

  These datasets help measure latency, synchronization accuracy, AI effectiveness, and user engagement.

3.9 Preprocessing of Code

Preprocessing ensures the system works efficiently:

- Validation: Detect syntax errors before execution
- Normalization: Standardize code formatting
- Error Handling: Identify potential runtime issues

3.10 Testing and Evaluation Sets

The system is tested using three sets:

- Training Set: Programs used to train the AI module
- Validation Set: Programs used to tune AI suggestions and real-time features
- Testing Set: Independent programs used to evaluate system performance and user experience

3.11 AI Module Implementation

The AI module uses pre-trained models or APIs such as OpenAI Codex to provide:

Chapter 4

Designing of Project

4.1 0-Level DFD

Figure 4.1 enlists the general flow of a brain tumor detection system, considering the general interaction between patients, doctors, medical equipment, and the system itself.

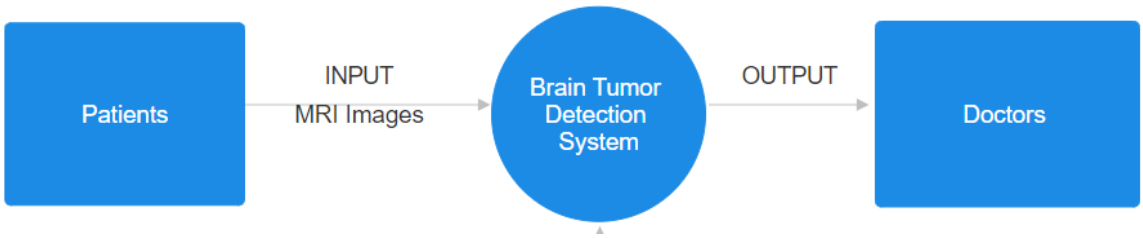

Figure 4.1: 0-level DFD

1. Patients : This process is started on patients who require a diagnosis of brain tumors. Patients are subjected to MRI scans, which remain the input to the detection system of brain tumors. The MRI images become vital data in identifying and even classifying the brain tumors.

2. Brain Tumor Detection System : This is the base system of the entire process. It incorporates MRI images, processes the images, and feeds algorithms associated with machine or deep learning techniques, including CNNs, to discern whether or not an individual has a brain tumor. If a tumor exists, then it further categorizes the type as malignant or benign.
- The system relies on medical equipment for quality images from MRI. The machinery produces images that are clear and accurate,

thereby giving a correct diagnosis. From the analysis, the system produces output, showing the existence of a tumor, classification, size, position, among other relevant facts.

3. Doctors: The output obtained by the brain tumor detection system is communicated to doctors. In this regard, it provides health care professionals with sufficient information to be able to make an appropriate decision toward the diagnosis and treatment of the patients. The doctors can utilize the outcome to diagnose the condition, develop treatment strategies, and provide proper prognosis to the patients.

4.2 1-Level DFD

Figure 4.2 is the workflow of developing a brain tumor detection and classification model with Logistic Regression (LR).

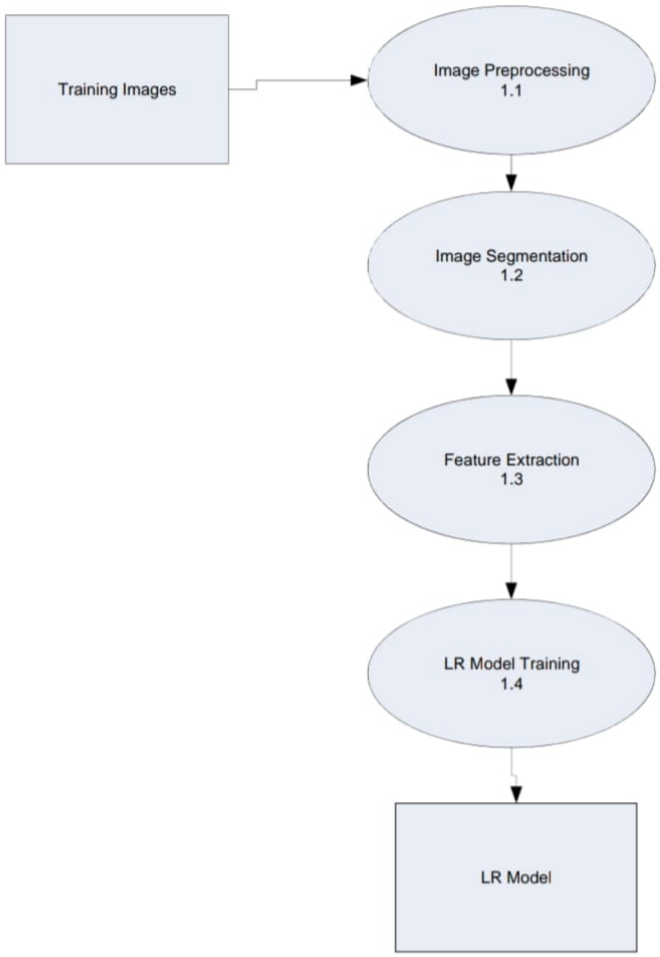

Figure 4.2: 1-level DFD

It is segmented mainly into four phases; therefore, it can be accordingly divided into four blocks: Image Preprocessing, Image Segmentation, Feature Extraction, and LR Model Training; hence, the resulting product is the trained Logistic Regression (LR) Model. Here are the details in detail:

1. Image Pre-processing (1.1) : This is a training image preparation stage. It falls into the operation stage whereby the following operations may be included: resize, normalization, contrast enhancement, and noise reduction which ensure that the quality in the images is maximized. These operations take the input data into one standard, thus appropriate for further processing and overall accuracy with the model.

2. Image Segmentation (1.2) : It segments the images after preprocessing. The regions of interest, in this case, include the tumor

areas, which get segregated from the rest of the brain tissues during the segregation procedure. Separation is a process that isolates the tumor for further study. The techniques include thresholding and region-based segmentation or edge detection to define their boundaries.

3. Feature Extraction (1.3): After segmentation of the tumor areas, key features are to be extracted from such areas. Feature extraction is defined as the process of quantifying important characteristics such as: Tumor size, shape, texture, and intensity. These features are used as the input by the classification model. Feature extraction transforms image data into numerical values describing the tumor;

4. Training the LR Model (1.4): This stage requires utilization of the extracted features to train the Logistic Regression (LR) model. This example does learn the logistic regression model on a labeled dataset to map input features with output classes. For brain tumor classification, the logistic regression model learns how to classify the two major types of tumors: benign and malignant by their respective features extracted.

4.3 2-Level DFD

In Figure4.3even refers to the full workflow diagram of the modified ResNet50 model, mostly for detection and classification of brain tumors. It follows this discussion up to the classification of brain tumors from data preprocessing.

1. Watershed Enhanced Segmentation

Better watershed segmentation as it separated the tumor from the background of the brain. Then, morphological restoration and markup extraction that fine-tuned the result image such that it was well segmented for the tumor.

2. Data Set Creation Warehouse the segmented images in a dataset. The dataset shall be applied when training and testing the machine learning model.

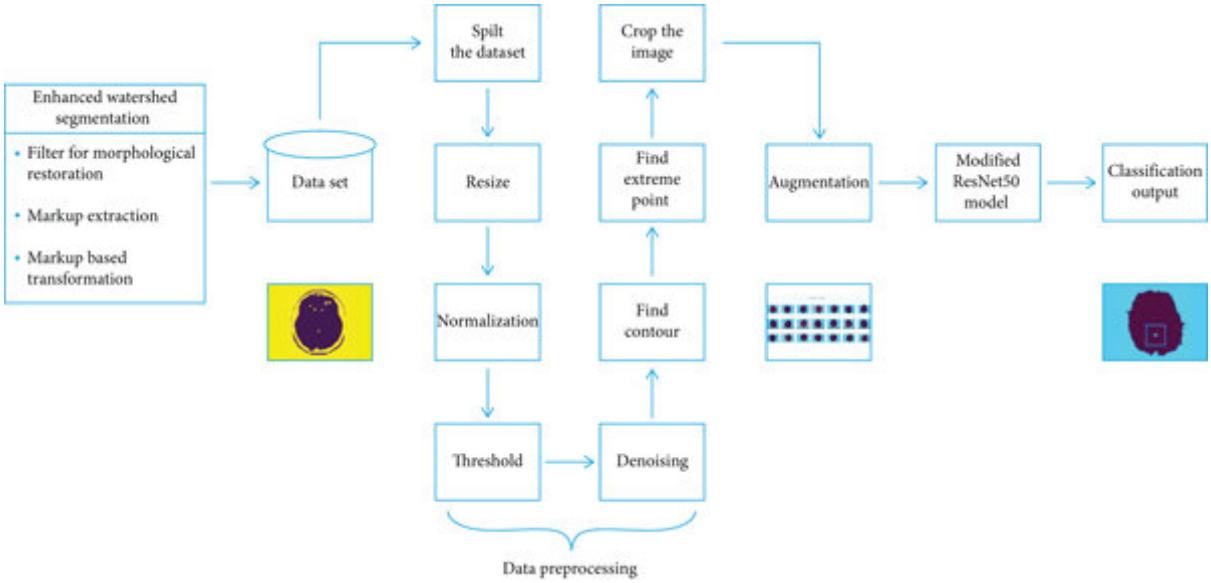

Figure 4.3: 2-level DFD

3 Preprocessing

All the images are resized to have uniformity as per the following size. Standardises pixel intensity values and quality enhancements in the images by removing the variations captured in the images,
Thresholding and Denoising: This would remove the unwanted noises and artifacts in the images that would be needed or not, therefore giving prominence to the regions of the tumor.
Contour Detection : This will enable tracing the contours of the tumor that existed in the brain image .
Extreme Points Detection : It will help detect the extreme points of the tumor so that proper image segmentation occurs.

4. Image Augmentation

It artificially increases the dataset by having techniques applied in the augmentation of data. This will provide the ability to generalize through making variations within data, for instance through rotation, scaling, or flipping.

5. ResNet50 Model The preprocessed and augmented data will be passed to the modified version of the ResNet50 model-it's really deep network-densely applied into the problems with more than two classes, like image classification but not limited to that. The model is trained on the task of tumour classification and detection.

6. Classification Output : Finally, it outputs the classification result as benign or malignant by using the features obtained from images. Workflow This workflow achieves the preciseness of classi-

fication for a given brain tumor through the process that combines preprocessing, segmentation, and deep learning.

4.4 Use Case Diagram

Below Figure 4.4depicts a use case for a system of brain tumor detection and classification involving two main actors, namely the Developer and the User .

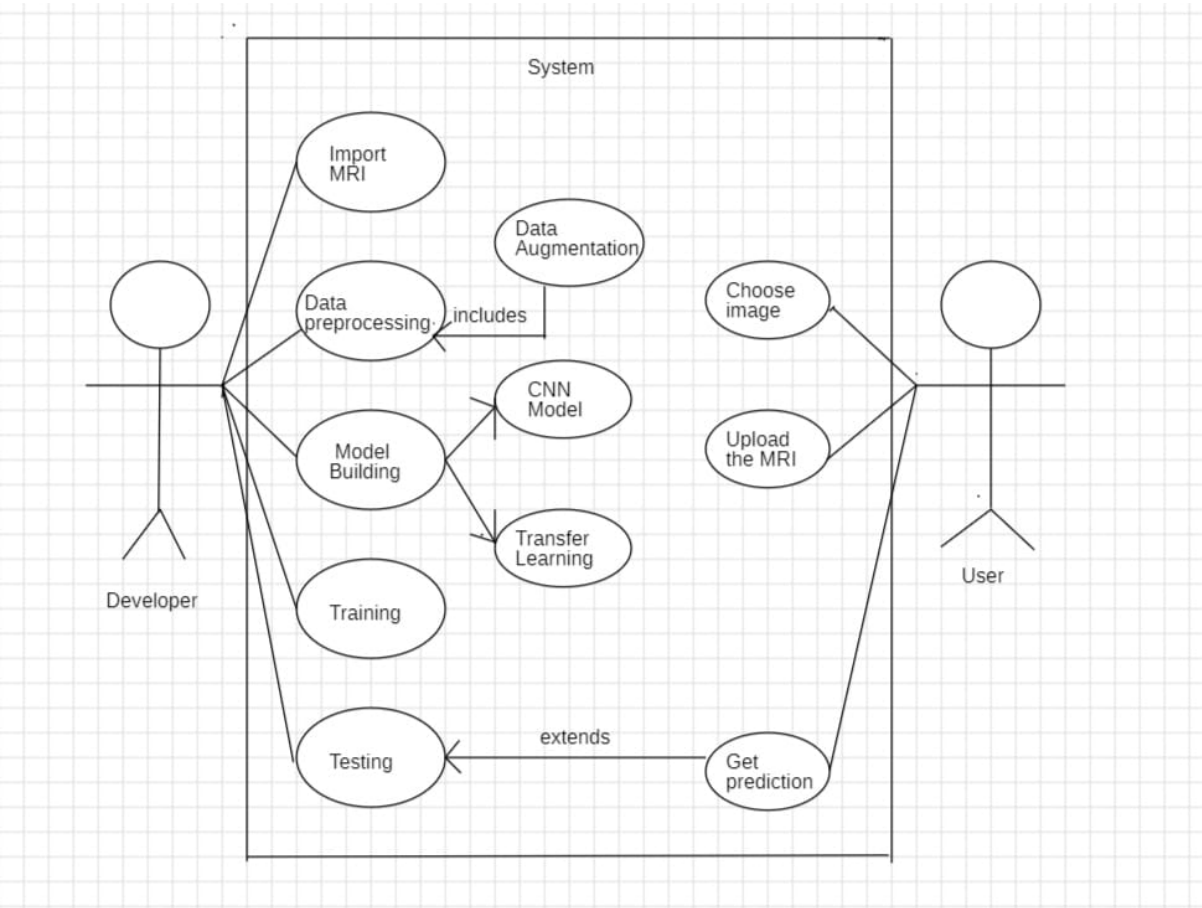

Figure 4.4: Use Case Diagram

1. Role of Developer

Developer deals with the back-end of the system. Such big tasks are:

- **Input:** MRI images This sets the system to accept the MRI images created by it.
- **Data Preprocessing** Cleaning, normalization, and enhancement of images for good quality data.

- **Feature Extraction :** Relevance features from the MRI images, such as size, texture, or intensity, have been among the chief factors for classification.
- **Training :** It is trained with features retrieved by machine learning algorithms like deep models.
- **Feature Matching :** The system, after being trained, uses learned features to classify and detect the tumours after matching newly input images.

2. Users Role

The user then interacts with a front-end interface, which is simpler to use for the system. Some of the most critical activities in this regard are:

- **Select Image :** The patient should be required to select an MRI image for the input analysis.
- **Upload MRI Image :** The system should accept an uploaded MRI image from the patient.
- **Get Prediction :** The user will get the prediction that the system computes based on its model as to what kind of tumor, whether it is benign or malignant, is going to be.

Chapter 5

Detailed Designing of Project

5.1 Class Diagram

The following Figure 5.1represents such a detailed workflow and class structure of a supposed system to automatically detect and classify brain tumors from MRI images.Such an intended system can be broken into primary categories: it includes pre-processing, feature extraction, segmentation, and classification, respectively. Further elucidation is needed in describing the steps of components and the interrelationship between these. Class of interface allows the user to access the system. The major methods include:

- **loadMRIImage(input) :** Loads the MRI image of the brain.
- **getTypeOfTumor() :** It returns the type of classification result determines which of the tumors is benign or malignant.
- **getSegmentedTumor():** This serves to return the segmented area in the MRI scan where the tumor is located.

1. Pre-processing : Image enhancement stage- here, most techniques enhance the quality of images and also include:

- **Skull Stripping :** Remove non-brain tissues, such as the skull to focus more on the regions of the brain.
- **Normalization :** It is mainly done to standardize image intensities for better analysis
- **Denoising :** Noise in MRI images is reduced such that clearer identification of higher features will be provided

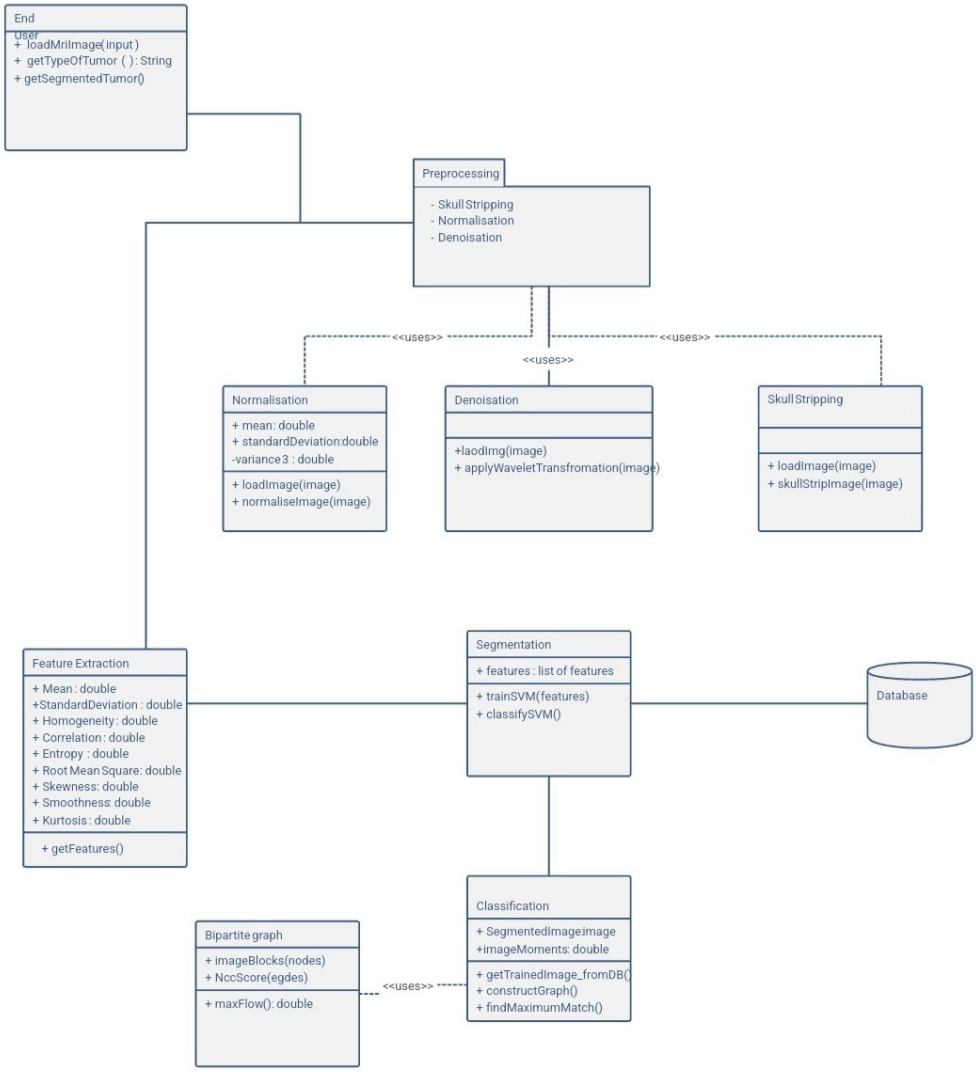

Figure 5.1: Class Diagram

2. Feature Extraction : After the pre-processing, the key features of the MRI image are extracted. The features include the statistical parameters that are: **Mean, Standard Deviation, Entropy, Skewness**, and much more giving a description regarding the texture and shape of a possible tumor. Input to classification model.

3. Segmentation : Break the image into regions of interest by segregating abnormal tissues (tumors) from normal brain tissues. This is through

SVM : It is a learning algorithm that gets to work on extracted features and then it will be capable of classifying regions to belong to either one class as tumor and the other as non-tumor.

4. Classification : It determines whether the identified tumor is either Benign or Malignant. And it is achieved through:

- Drawing up a bipartite graph based on the segmented images.
- Maximal Match classification-identify the type of tumor by retrieved stored models in a database.

Briefly speaking, the system works step by step from preprocessing of the brain MRI image, feature extraction, segmentation, and classification for correct results in the detection and classification of brain tumors. In this paper, statistical analysis along with various machine learning techniques like SVM plus graph-based classification have been performed for robust performance.

5.2 Entity-Relationship Diagram (ERD)

This figure 5.2attached image is the Entity-Relationship (ER) Diagram of a system that detects and classifies brain tumors.

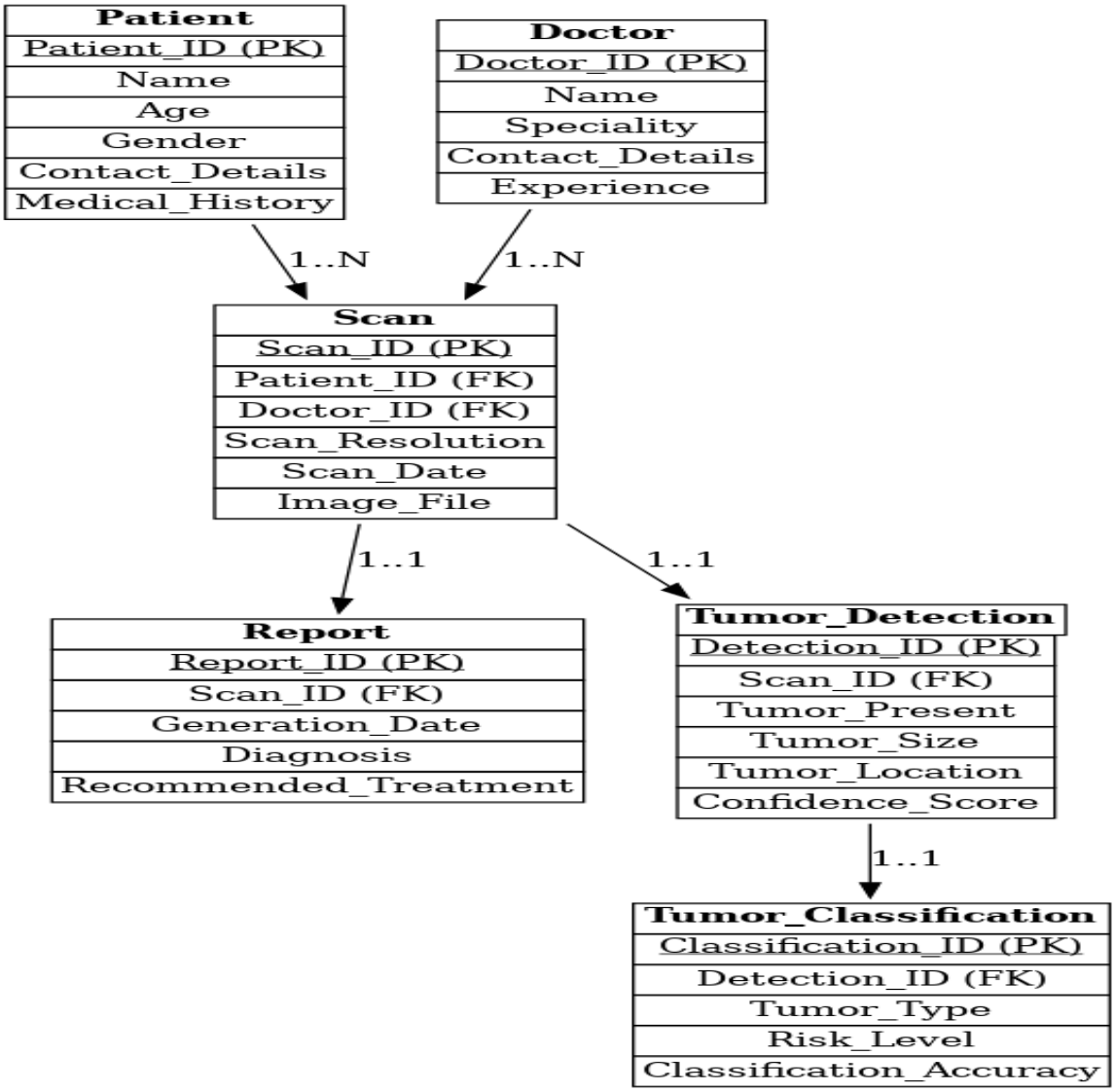

Figure 5.2: ER Diagram

It is actually the description of the relations which may exist between several entities in the process, like patients, doctors, scans, and diagnostic results.

1. Patient Attributes: Patient ID, Name, Age, Gender, Contact Details, Medical History.

Medical History of the patient is noted and scanned times, and he is diagnosed accordingly.

2. Doctor : Attributes : Doctor ID, Name, Speciality, Experience, Contact Details. Diagnose for the scan is selected that comes with reading of multiple reports prepared by various doctors.

3. Scan : Attributes: Scan ID, Scan Resolution, Scan Date, Image File. Every Scan is provided to the patient and results in association

with classification and detection of tumor.
4. Detection of Tumor : Attributes: Detection ID, Tumor Present, Tumor Size, Tumor Location, Confidence Score. This is also kept there to state if a tumor exists or not and then combined with other information like its size, location, and even the extent to which one believes the system will be accurate in detecting.
5. Tumor Classification : Attributes: Classification ID, Tumor Type, Risk Level, Classification Accuracy. Once there is a tumor, the system will diagnosis if the tumor is benign or malignant and its likely chance of the cancer it has and also gives an accuracy classification of the cancer.
6. Report : Attributes: Report ID, Generation Date, Diagnosis, Recommended Treatment. This report would be produced based on the results of the scan and the classification of the tumor as a diagnosis along with appropriate treatment recommendations if needed.

5.3 Activity Diagram

In this figure5.3 schematic explains the process by which a multi-step machine learning technique performs the task of detection and classification of brain tumors from an MRI image. It has different significant steps on its workflow:
1. Brain MRI Image Acquisition : It first acquires MRI scans of the brain, which are basically provided as input to the system.
2. Pre-processing: The MRI images are now preprocessed in a bid to enhance the quality of images so acquired by removing noise, normalization of data as well as enhancing their contrasts for better reputation fit to serve purposes of analysis.

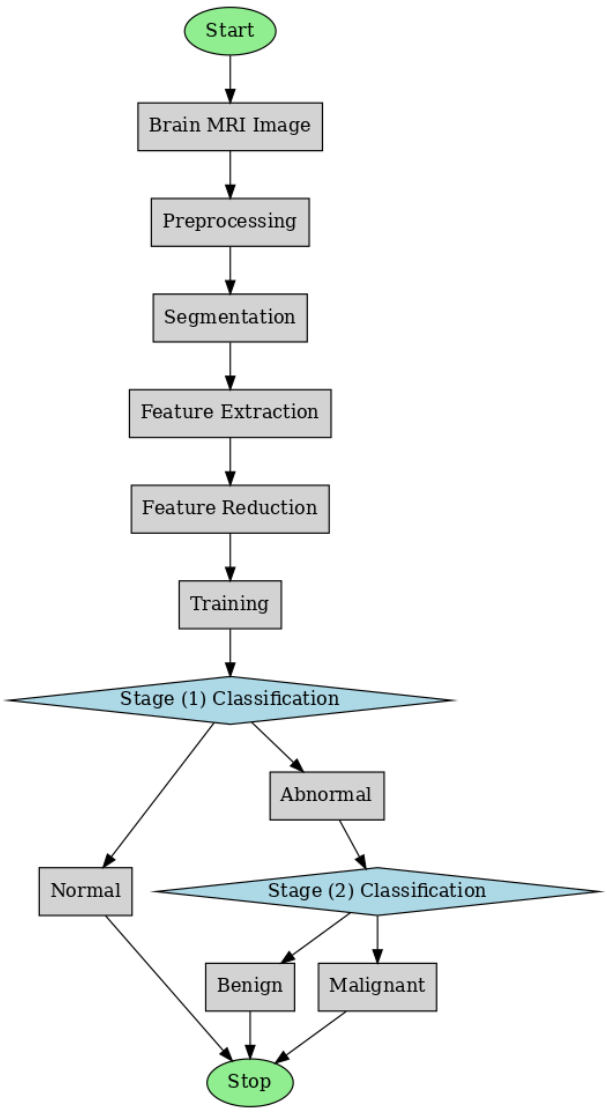

Figure 5.3: Activity Diagram

3. Segmentation : In this phase, the picture or image gathered from the brain is segmented. Thus, regions containing probable tumours are separated from other tissues. In this segmentation process, regions of interest are defined based on which the tumors propagate themselves.

4. Feature Extraction : In the case at hand, the relevant characteristics are the texture, shape, or size in the regions of interest after segmentation that would describe the normal tissues and the abnormal tissues.

5. Feature Reduction : In this step, the redundancy and irrelevance features are to be eliminated since in this process, it supposedly reduces the total number of features only to the ones that are most significant in ensuring accurate classification.

6. Training and Classification Phase 1: These features that are obtained are fed into the trained learning machine model that was pre-trained to classify the image as "Normal" or "Abnormal". If it classifies as "Normal", then no other process is called for. If it is classified as "Abnormal", then the given operation is passed on to the next phase.
7. Classification : Step 2 Such images can be classified as abnormal. These abnormalities are then subclassified into benign, which is non-cancerous, and malignant, which is cancerous classification. As classified, are thus used in guiding the treatment plans.

5.4 Sequence Diagram

The following diagram 5.4 gives a sequence flow in the detection of a brain tumour, using techniques both in image processing and machine learning.
There are phases, actually several within the process, from the original image up to the final result. For this answer, I elaborate step by step.

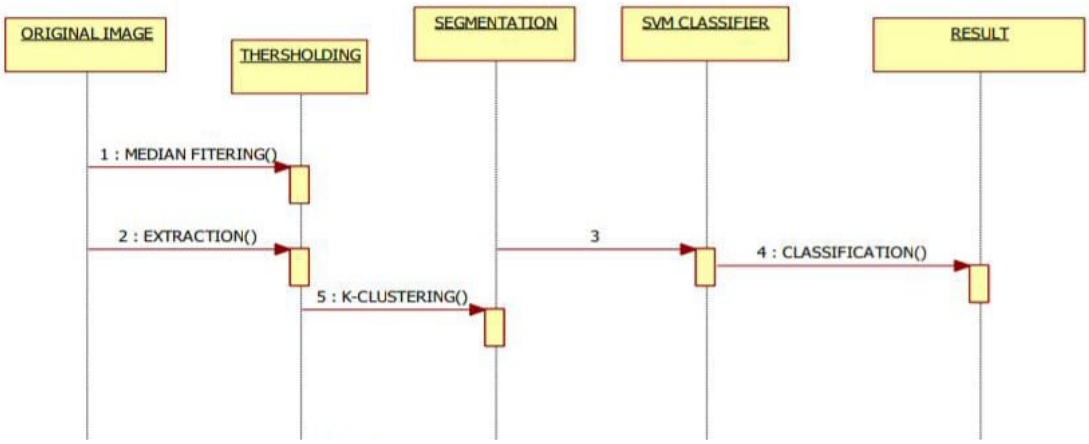

Figure 5.4: Sequence Diagram

1. Original Image : It starts with an MRI picture or picture of any other brain. That picture forms the raw data source to be fed in further for processing .
2. Thresholding : It is the first pre-processing step in which median filtering is applied on an image to remove noise. Noise removal

enhances the quality of the image by making it smooth without losing the essential features of the image, which are edges. - Extraction : Features of interest or the region of interest is extracted from the image after filtration. Such regions might be potential tumors and hence are focused upon to extract them.

3. Segmentation : This algorithm implements the usage of K-Means clustering with which it clusters the image. In the process, it groups the pixels into various clusters based on the resemblance, to separate the tumor area from the normal brain tissue. This step of segmentation would very well help to isolate the region of tumor to further analyze.

4. SVM Classifier : From this classification, the classified image is transferred to the SVM classifier, which in reality uses SVM. SVM is one of the binary classification algorithms which applies data-driven rules in a decision to classify as benign or malign.

5. Result : Such an output will be generated based upon the prediction that SVM has made in the result and therefore, it will come up with the output as benign tumors or malignant tumors. Therefore, such a type of output will further deliver the result towards the user for diagnosis and treatment planning in the medicine field.

Appendix A

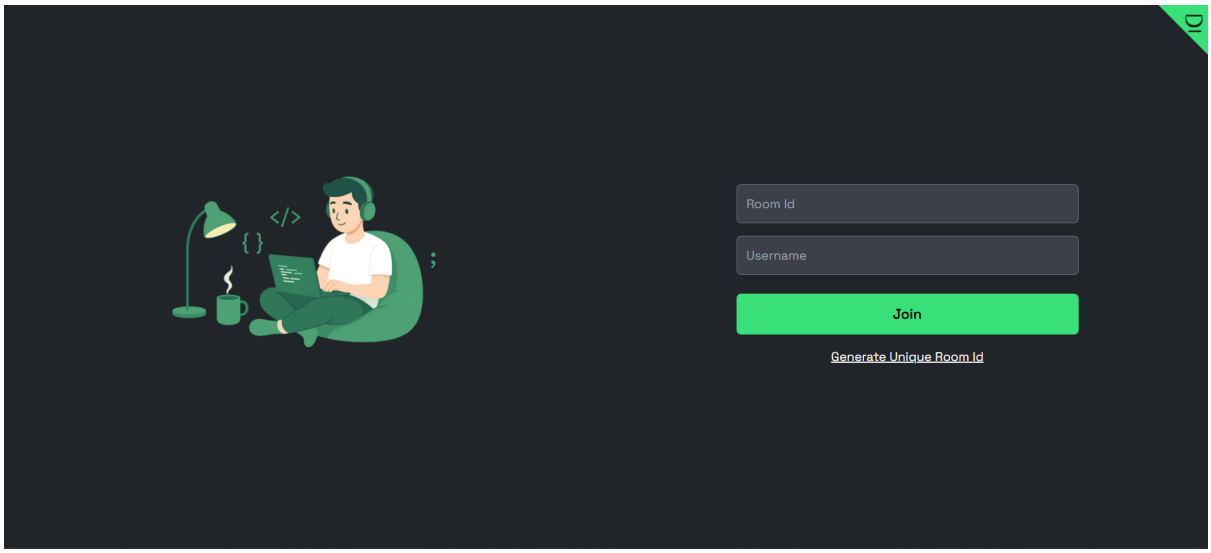

Figure 5.5: Screenshot Of Application

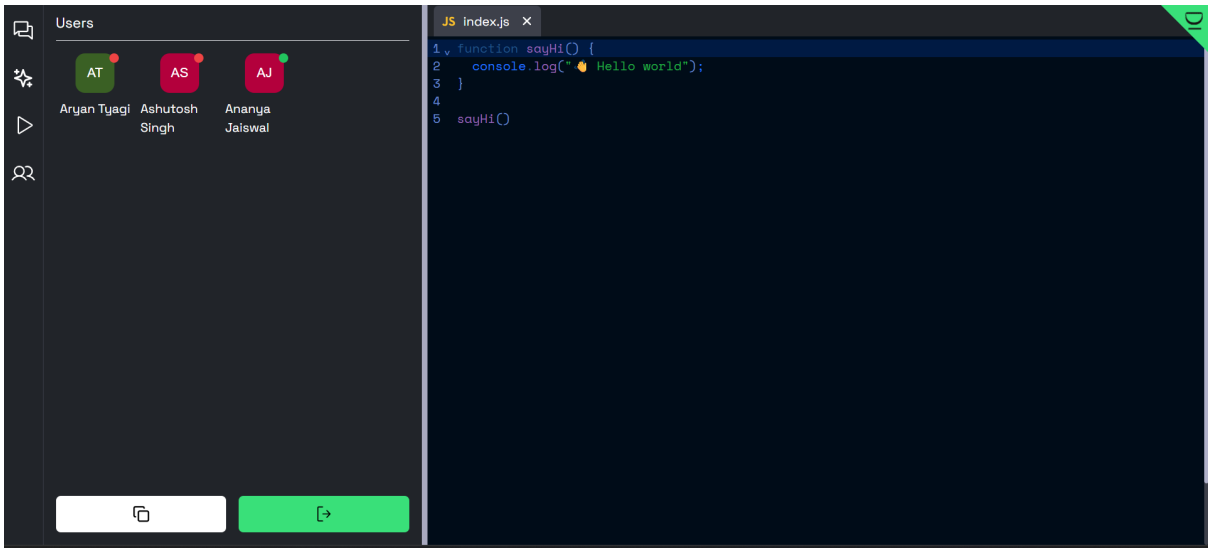

Figure 5.6: Screenshot Of Application

Appendix B

Real-Time Code Editor

Ananya Jaiswal
Information Technology
Ajay Kumar Garg Engineering College
Ghaziabad, Uttar Pradesh
ananya2213085@ akgec.ac.in

Ashutosh Singh
Information Technology
Ajay Kumar Garg Engineering College
Ghaziabad, Uttar Pradesh
ashutosh2213185@ akgec.ac.in

Aryan Tyagi
Information Technology
Ajay Kumar Garg Engineering College
Ghaziabad, Uttar Pradesh
aryan2213184@ akgec.ac.in

***Abstract*—Coding education has become a global priority, yet many school systems struggle to provide students with meaningful exposure to real-world programming practices. Traditional curricula often emphasize syntax drills rather than collaborative, project-based learning, while school labs are limited by outdated infrastructure, expensive software, and steep learning curves. This paper proposes the design and development of a real-time collaborative online code editor tailored for school students, addressing both technical and pedagogical gaps. The platform enables students to code together synchronously through a web browser without installation or mandatory accounts, integrates an AI assistant for contextual guidance, and implements screen-time notifications to encourage healthy digital habits. We review recent research on collaborative editing frameworks, computational thinking in K–12 education, cloud-based IDEs, and AI tutoring systems. Building on these insights, we propose an architecture based on conflict-free replicated data types (CRDTs) and lightweight web technologies, with a focus on accessible UI/UX for novice learners. The proposed solution is expected to improve engagement, collaboration, and inclusivity in coding education, while highlighting new challenges around teacher training, equity, and ethical AI use.**

***Index Terms*—Collaborative Code Editor, Artificial intelligence, Chat Bot, Natural Language Processing**

I. Introduction

In the last decade, computational literacy has been increasingly recognized as a foundational skill for students in the digital age. With artificial intelligence, data science, and automation influencing nearly every industry, schools are under pressure to equip students with programming knowledge that extends beyond theory into authentic practice. Yet, despite policy changes in several countries, including the integration of Python, Java, and Scratch into secondary curricula, the exposure that students receive to real-world coding practices remains limited [1]. Many schools, especially in low-resource settings, face challenges such as outdated computer labs, lack of access to personal devices, insufficiently trained teachers, and reliance on software that requires installation and steep learning curves [2].

The problem is further compounded by socioeconomic disparities. Students whose parents cannot afford personal laptops or mobile devices often depend entirely on school computer labs [3]. These labs are frequently constrained by outdated infrastructure, restricted internet access, or licensing issues that prevent schools from using modern development environments [4]. As a result, coding education in schools remains largely theoretical, limiting students' ability to engage in hands-on, project-based learning, which is critical to developing computational thinking and problem-solving skills [5].

Simultaneously, the digital generation is growing up surrounded by collaborative technologies. Students are accustomed to real-time communication through messaging apps, multiplayer gaming platforms, and collaborative tools such as Google Docs. However, these same collaborative paradigms are rarely applied to programming education [6]. Existing educational IDEs such as Scratch and Code.org offer block-based coding but lack the depth and real-world relevance of text-based programming environments [7]. Meanwhile, professional tools like Replit or Visual Studio Live Share, while powerful, often present interfaces that can overwhelm beginners and require sign-ups or installations, creating barriers to adoption in schools [8].

This gap underscores the need for a solution that merges real-time collaboration, simplicity, and accessibility. A web-based code editor that runs directly in the browser without installation or accounts can eliminate many entry barriers for students. The platform should also embrace a collaborative-first approach, allowing multiple students to edit, debug, and build projects together in real-time, mirroring the way professional developers work in distributed teams [9]. Such an approach supports both peer learning and teacher-guided classroom activities.

An additional opportunity lies in the integration of AI-powered tutoring. Large Language Models (LLMs) and specialized code assistants can provide explanations, debugging hints, and personalized feedback at scale [10]. While these tools cannot replace teachers, they can democratize access to coding support, especially in schools where teacher training is limited. At the same time, safeguards must be established

Figure 5.7: Research paper

Appendix C

Figure 5.8 shows the plagiarism report of our project, which is 8 percentage

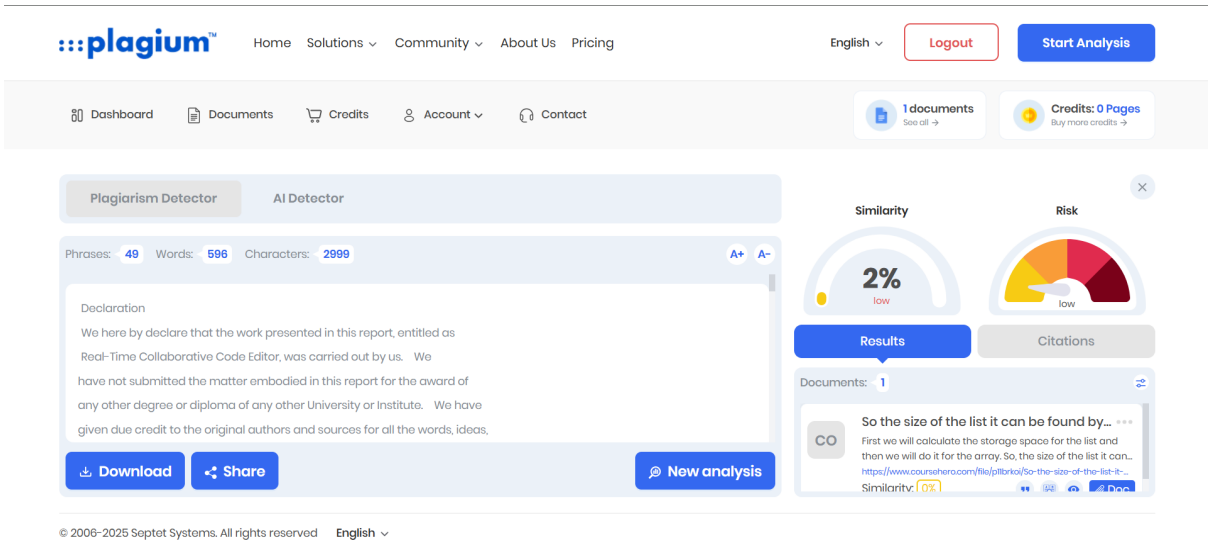

Figure 5.8: Plagiarism Report

Figure 5.9 shows the AI report of our project, which is less than 20 percentage

8%	15%	4%	7%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Kongu Engineering College Student Paper	1%
2	C. Edwards, I.I. Rahman. Collaborative Real-Time Programming in Python Using Web Sockets. Proceedings of ICETET 2023 Internet Source	1%
3	www.researchgate.net Internet Source	<1%
4	cloud.google.com Internet Source	<1%
5	M. Coblenz, Jupiter: Programming as a Service for Efficient and Scalable Cloud Code Editing, UIST 2017 Publication	<1%
6	M. Bajammal, K. Deshmukh, S. Salvi, R. R. Negenborn. Collaborative Code Editing: Web-based Software Development through Smart Devices, Springer, 2020	<1%

Figure 5.9: AI Report